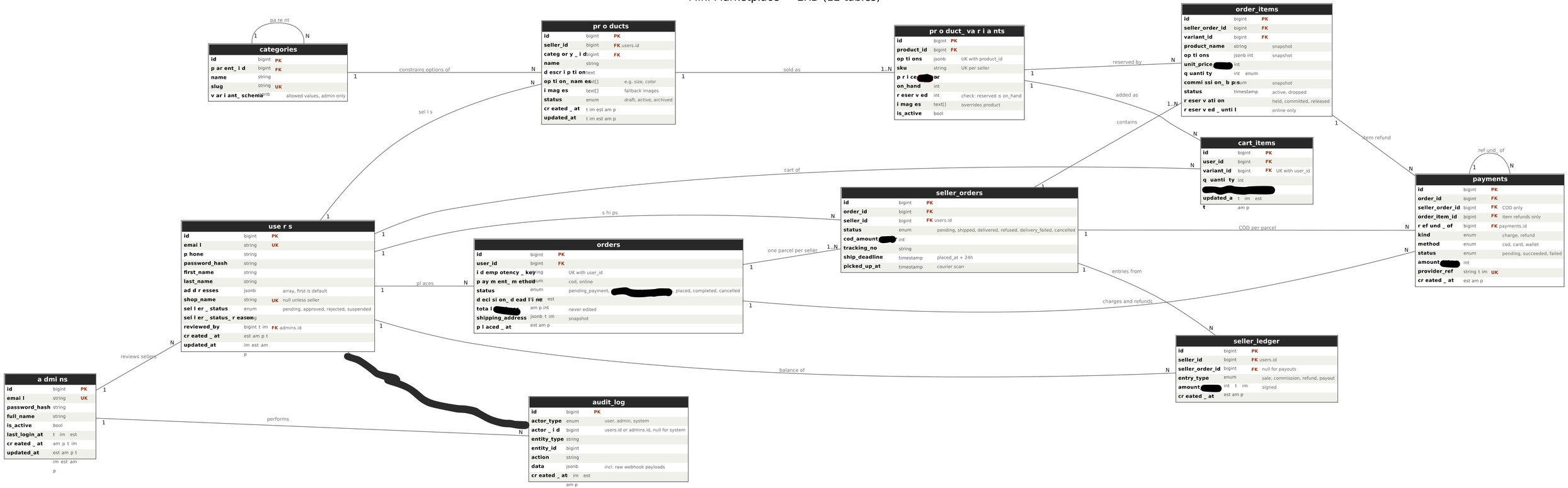

Mini Marketplace — ERD (12 tables)
categories
id bigint PK
parent_id bigint FK
name string
slug string UK
variant_schema jsonb allowed values, admin only
parent
products
id bigint PK
seller_id bigint FK users.id
category_id bigint FK
name string
description text
option_names text[] e.g. size, color
image_urls text[] fallback images
status enum draft, active, archived
created_at timestamp
updated_at timestamp
constrains options of
sells
sold as
product_variants
id bigint PK
product_id bigint FK
option_values jsonb UK with product_id
sku string UK per seller
price_cents int
on_hand int check: reserved ≤ on_hand
image_urls text[] overrides product
is_active bool
reserved by
added as
order_items
id bigint PK
seller_order_id bigint FK
variant_id bigint FK
product_name string snapshot
option_values jsonb snapshot
unit_price_cents int
quantity int
commission_bps int snapshot
status timestamp active, dropped
reservation_id held, committed, released
reservation_until online only
item refund
contains
cart of
cart_items
id bigint PK
user_id bigint FK
variant_id bigint FK UK with user_id
quantity int
updated_at timestamp
users
id bigint PK
email string UK
phone string
password_hash string
first_name string
last_name string
addresses jsonb array, first is default
shop_name string UK null unless seller
seller_status enum pending, approved, rejected, suspended
seller_status_reason string
reviewed_by bigint FK admins.id
created_at timestamp
updated_at timestamp
places
shops
seller_orders
id bigint PK
order_id bigint FK
seller_id bigint FK users.id
status enum pending, shipped, delivered, refused, delivery_failed, cancelled
cod_amount_cents int
tracking_no string
ship_deadline timestamp placed_at + 24h
picked_up_at timestamp courier scan
one parcel per seller
COD per parcel
orders
id bigint PK
user_id bigint FK
idempotency_key string UK with user_id
payment_method enum cod, online
status enum pending_payment placed, completed, cancelled
delivery_deadline timestamp
total_cents int never edited
shipping_address jsonb snapshot
placed_at timestamp
payments
id bigint PK
order_id bigint FK
seller_order_id bigint FK COD only
order_item_id bigint FK item refunds only
ref_und_of bigint FK payments.id
kind enum charge, refund
method enum cod, card, wallet
status enum pending, succeeded, failed
amount int
provider_ref string UK
created_at timestamp
ref_und_of
entries from
charges and refunds
seller_ledger
id bigint PK
seller_id bigint FK users.id
seller_order_id bigint FK null for payouts
entry_type enum sale, commission, refund, payout
amount int signed
created_at timestamp
balance of
admins
id bigint PK
email string UK
password_hash string
full_name string
is_active bool
last_login_at timestamp
created_at timestamp
updated_at timestamp
reviews sellers
performs
audit_log
id bigint PK
actor_type enum user, admin, system
actor_id bigint users.id or admins.id, null for system
entity_type string
entity_id bigint
action string
data jsonb incl. raw webhook payloads
created_at timestamp

Mini Marketplace ERD

Overview

this document explains every decision in the mini marketplace ERD and why we made it. the marketplace follows the Daraz model: one user can both buy and sell, one cart holds items from many sellers, and each seller ships their own parcel through Leopards Courier. customers pay online (card or wallet) or cash on delivery.

the ERD has 12 tables grouped into 6 areas. table names are written in quotations and columns in code formatting.

Area	Tables
Identity	"users", "admins"
Catalog	"categories", "products", "product_variants"
Cart	"cart_items"
Orders	"orders", "seller_orders", "order_items"
Payments and money Infra	"payments", "seller_ledger"
	"audit_log"

three rules apply across every table:

- **money is stored in paisa as whole numbers.** `price`, `total`, `amount` and every other money column is an integer, so Rs 1,299 is stored as 129900. the column names don't mention paisa, so this rule is where the unit is defined. floats can't represent most decimal amounts exactly and rounding errors pile up over thousands of orders.
- **anything an order depends on is copied at checkout.** product name, options, unit price, commission rate and shipping address are copied (snapshotted) onto the order. a seller editing a product or a customer editing an address later never changes an order that already happened.
- **fixed sets of values are enums.** statuses, methods and types are enums so the database itself rejects a value we never planned for.

Identity

customers and sellers share one table named "users", and admins have their own table named "admins".

"users"

- **one table for customers and sellers.** a user signs up as a customer by default with null seller related fields (`shop_name`, `seller_status`, `seller_status_reason`, `reviewed_by`) and then registers as a seller later. keeping them separate caused a redundant table with duplicated data (email, phone, passwords etc.) and meant one person needed two logins to buy and to sell.
- **seller registration is reviewed by an admin.** `seller_status` moves through pending, approved, rejected and suspended. `seller_status_reason` stores why a seller was rejected or suspended so we can tell them, and `reviewed_by` records which admin made the call.
- **`shop_name` is unique** so two shops can't share a name. it stays null unless the user is a seller.
- **`email` is unique** because it is the login.
- **we store `password_hash`, never the password itself**, so a leaked database doesn't leak passwords.

"admins"

- **admins have their own table for security reasons.** if there is a bug in the user auth that lets a user give themselves extra privileges, they still can't become an admin, because admin accounts live in a separate table with a separate login.
- **`is_active` disables an admin without deleting them.** deleting would break `users.reviewed_by` and the admin's history in "audit_log".
- **`last_login_at`** helps spot admin accounts that are unused or being used when they shouldn't be.

addresses

- **addresses are stored as a list within the "users" table** (`addresses`, a jsonb array). the first address in the list is the default.

- **nothing else references an address.** at checkout the chosen address is copied into `orders.shipping_address`, so no other table ever needs an address id. a separate addresses table would only add a join for no benefit.

Catalog

a seller lists a product under a category, and the product's sizes and colours are separate rows in "product_variants", each with its own price, SKU and stock.

"categories"

- **`parent_id` allows sub-categories.** it points to another category, so Phones, Laptops and Desktops can be sub-categories of the Electronics category. a top-level category has no parent.
- **`slug` is unique** so each category has its own clean URL.
- **`variant_schema` holds the allowed options for the category**, e.g. clothing allows sizes S, M, L and colours red, blue, black. only admins can edit it. we kept options as a list created by admins rather than having sellers enter those values themselves to avoid mixed values or situations where "red" and "Red" are different. it also means a colour filter works the same across every seller.

"products"

- **a product holds what all its variants share:** seller, category, name, description and images.
- **`option_names` picks which options from the category's schema this product uses.** a T-shirt might use size and colour while a phone case only uses colour.
- **`status` is draft, active or archived.** sellers can prepare a listing before publishing it, and archive it instead of deleting it, because old orders still point at it.
- **`images` are the fallback images** shown when a variant has none of its own.

"product_variants"

- **a separate table to group products according to size/colour.** each variant has its own price, SKU, stock and option values (`options`, picked from the admin-created list).
- **`options` is unique together with `product_id`** so one product can't have two "Red, M" variants.

- **`sku` is unique per seller, not across the platform**, because two different sellers can both use "TSHIRT-01".
- **stock lives on the variant instead of a separate stock table**, since each variant has exactly one stock count. `on_hand` is how many units the seller has and `reserved` is how many are held for orders that aren't confirmed yet. available stock is `on_hand` minus `reserved`. a check makes sure `reserved` can never exceed `on_hand`, so the database itself stops us overselling.
- **variant `images` override the product's `images`**, so the red shirt shows red photos.
- **`is_active` hides one variant** (e.g. a size the seller no longer makes) without archiving the whole product.

Cart

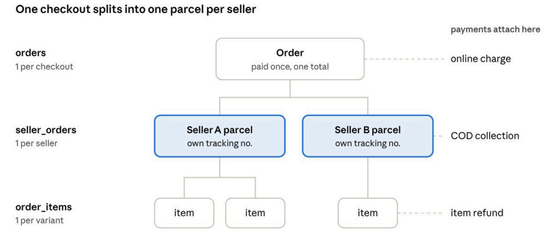
each user's cart is just their rows in "cart_items".

- **we removed the separate "cart" table to reduce redundancy** because all it did was show "cart_items". every user has exactly one cart and there are no guest carts, so `cart_items.user_id` is enough to find a user's cart.
- **`variant_id` is unique together with `user_id`.** adding the same variant again raises the `quantity` instead of creating a duplicate row.
- **`selected` supports selective checkout.** the customer ticks the items they want to buy now, and the rest stay in the cart for later.
- **no price is stored in the cart.** the price is read from the variant every time, so the cart always shows the current price. the price only gets locked at checkout, in "order_items".

Orders

one checkout creates one row in "orders", one row in "seller_orders" per seller in it, and one row in "order_items" per variant bought.

one cart holds items from different sellers, and each seller ships an order from their end, so the tracking number, amount, deadline etc. have to exist per seller. hence we added a separate table "seller_orders" between "orders" and "order_items".



order split · 1 order, 2 seller parcels, 3 items

each seller's parcel ships, gets paid for (COD) and gets tracked on its own, while the customer still sees one order.

"orders"

- **one row is what the customer placed and pays for**, however many sellers it covers.
- **`idempotency_key` is unique together with `user_id`.** if the checkout request is sent twice (a double click or a network retry) the second one creates nothing. it is unique per user, not across the platform, because the key is generated on the customer's side.
- **`payment_method` is `cod` or `online`** and is chosen once for the whole order.
- **`status` is `pending_payment`, `placed`, `completed` or `cancelled`.** an online order waits in pending_payment until the gateway confirms, while a COD order is placed straight away. an order is completed once its parcels are delivered.
- **`total` is never edited.** it is what the customer agreed to pay at checkout. if an item is dropped later, the refund is recorded in "payments" and the total stays as it was, so the order always matches the customer's original receipt.
- **`shipping_address` is a snapshot** copied from the user's address list at checkout.

"seller_orders"

- **one row is one seller's parcel.** each one has its own status, tracking number and deadline.

- `status` **is pending, shipped, delivered, refused, delivery_failed or cancelled.** refused means the customer turned the parcel away at the door, and delivery_failed means the courier couldn't deliver it.
- `ship_deadline` **gives the seller 24 business hours** from when the order is placed to hand the parcel to the courier.
- `picked_up_at` **is set when Leopards scans the parcel at pickup.** comparing it to `ship_deadline` shows which sellers ship late.
- `tracking_no` is the Leopards tracking number for this parcel.
- `cod_amount` **is the cash the courier collects for this parcel.** it is per parcel because each parcel is delivered, and paid for, separately.

"order_items"

- **"order_items" includes the items that a customer selected from his cart to checkout with**, one row per variant.
- `product_name, options, unit_price and commission_bps are snapshots` copied at checkout, so a seller editing their product later doesn't change the order history or what the seller is owed.
- **commission_bps is the platform's commission in basis points** (100 bps = 1%, so 1000 = 10%). we use one platform-wide rate, and copy it onto each item so if we change the rate later, old orders keep the rate they were sold at. basis points keep it a whole number.
- `status` **is active or dropped.** if a seller can't fulfil an item, we drop only that item and refund it, and the rest of the order goes ahead. customers can't cancel part of an order.

stock reservation

"order_items" tracks reservation state instead of creating a separate table for managing stock. `reservation` is held, committed or released:

1. at checkout, each item's quantity is added to its variant's `reserved` count and the item is held.
2. for an online order, the item is committed when the payment succeeds. if the payment fails or `reserved_until` passes first, it is released and the stock becomes available again.

3. a COD order has no payment to wait for, so its items are committed when the order is placed. this is why `reserved_until` is only used for online orders.
4. a dropped item is released.

Payments and money

all money between customers and the platform is recorded in "payments", and all money between the platform and sellers is recorded in "seller_ledger".

the platform holds the account with the payment gateway and the account with Leopards Courier, so the customer pays once for his whole order even when it includes products from multiple sellers. the platform then pays each seller what they are owed, minus commission.

"payments"

- **"payments" stores every money movement between a customer and the platform:** charges and refunds, including failed attempts.
- **what a payment links to depends on what kind of payment it is.** every payment has an `order_id`, and some also point to a parcel or an item:

Payment	Links to	Why
online charge (card or wallet)	`order_id`	the customer pays once for the whole order
COD collection	`order_id` + `seller_order_id`	the courier collects cash per parcel
item refund	`order_id` + `order_item_id`	a dropped item is refunded on its own

- **`kind` is charge or refund, and `refund_of` points to the charge being refunded.** every refund can be traced back to the payment it came from.
- **`method` is cod, card or wallet.**
- **`status` is pending, succeeded or failed.** a failed attempt stays as its own row so we keep a record of every attempt.

- **`provider_ref` is unique.** it is the gateway's transaction id. gateways can send the same confirmation more than once, and the unique ref stops one payment being recorded twice.

"seller_ledger"

- **"seller_ledger" keeps a track of any payments, dues, commissions between the platform and the sellers.**
- **each row is one entry with a signed amount.** `entry_type` is sale (+), commission (−), refund (−) or payout (−). a seller's balance is the sum of their entries.
- **entries are only added, never edited.** a mistake is fixed by adding a correcting entry, so every rupee in a balance can be explained.
- **`seller_order_id` ties a sale, commission or refund to the parcel it came from.** it is null for payouts because one payout covers many parcels.
- **we chose a ledger over a single balance column** because a balance column can drift out of sync and can't explain how it got its value.

Infra

background jobs run on graphile-worker, and every important event is recorded in "audit_log".

job queue

- **we don't design our own table for queuing jobs, we use graphile-worker instead.** it runs on the Postgres database we already have, so we don't need Redis or any other service.
- **graphile-worker keeps its jobs in its own schema (`graphile_worker`)**, which it creates and manages itself. that table isn't part of our ERD and we never write migrations for it.
- **a job can be added in the same transaction as the change that caused it.** if placing an order fails and rolls back, the jobs for that order are rolled back with it, so a job never runs for something that didn't happen. e.g. releasing reservations when `reserved_until` has passed, or sending order emails.

"audit_log"

- **"audit_log" table stores every event along with the actors involved and the date and time** for admins to monitor and track all activity on the platform.
- **actor_type is user, admin or system.** actor_id points to users.id or admins.id depending on the type, and is null for system events like jobs and webhooks.
- **entity_type and entity_id say what was changed** (e.g. a seller order or a product), and action says what happened to it.
- **data stores the details of the event, including raw webhook payloads** from the payment gateway and Leopards. if there is a dispute, we can check exactly what the provider sent us.
- **one log table covers every entity** instead of a separate history table for each one.